Velzenix Nova – Complete System Guide

1. Introduction

Velzenix Nova ATS is a recruitment management application.

The purpose of this application is to help companies manage their complete recruitment process.

Instead of maintaining candidate information in Excel sheets, emails, and separate documents, the application manages everything in one place.

It helps companies:

- Create client recruitment portals.
- Create recruitment requirements.
- Add Hiring Managers and Recruiters.
- Assign different roles to users.
- Receive candidate applications.
- Screen candidate resumes.
- Move candidates through recruitment stages.
- Approve or reject candidates.
- Put candidates on hold.
- Generate offer letters.
- Send offer letters through email.
- Allow candidates to accept or decline offers.
- Collect candidate details and documents.
- Verify candidate information through OTP.
- Allow HR to review documents.
- Request corrections.
- Approve pre-onboarding submissions.

PART 1 – UNDERSTANDING THE APPLICATION

2. How the Application Is Built

The application has three major parts.

2.1 Frontend – What users see

The frontend is the website that users open in their browser.

Technology: Angular.

Examples:

- Login page
- Admin dashboard
- Client dashboard
- Candidate list
- Candidate details
- Recruitment stages
- Offer management
- Pre-onboarding forms

The frontend displays information and collects user input.

2.2 Backend – What happens behind the scenes

The backend manages the actual application logic.

Technology: Java and Spring Boot.

Examples:

- Checking login credentials.
- Checking user permissions.
- Saving candidate information.
- Managing recruitment stages.
- Generating offer letters.
- Sending emails.

- Generating and validating OTPs.
- Checking uploaded documents.
- Saving approval history.

The backend receives requests from the frontend, processes them, and sends responses.

2.3 Database – Where application information is stored

Technology: MySQL.

The database stores information such as:

- Users
- Clients
- Roles
- Requirements
- Candidates
- Recruitment stages
- Candidate history
- Offer details
- Pre-onboarding information
- Document information

The database does not normally store the actual resume or uploaded document contents. Those files are kept in separate storage directories.

3. How the Three Parts Communicate

Example: A Recruiter creates a candidate.

1. Recruiter opens the candidate creation page.
2. Recruiter enters candidate information.
3. Angular sends the information to the backend.
4. Spring Boot receives the request.
5. Backend checks the user's login and permissions.
6. Backend validates the candidate information.
7. Backend saves the information in MySQL.
8. Backend returns a successful response.

9. Angular displays the newly created candidate.

Simple architecture:

Browser → Angular Frontend → Spring Boot Backend → MySQL Database

For file uploads:

Browser → Angular → Backend → Private File Storage

For emails:

Backend → Email Service → Candidate's Email Inbox

PART 2 – APPLICATION DATA STORAGE

4. Where Is Data Stored?

The application uses two main types of storage.

4.1 MySQL Database

MySQL stores structured information.

For example, when a candidate is created, the database stores:

- Candidate name
- Email
- Phone number
- Requirement ID
- Client ID
- Experience
- CTC
- Notice period
- Skills
- Current recruitment stage
- Candidate status

4.2 File Storage

Files are stored separately from normal database records.

The application uses these default directories:

Directory	Purpose
`uploads`	General application uploads
`private-resumes`	Candidate resumes
`private-preonboarding`	Candidate pre-onboarding documents

The database stores the file information and reference. The actual file is stored in the corresponding directory.

4.3 Example: Candidate Resume

Suppose a Recruiter uploads:

`xyz`

The process is:

1. Recruiter selects the resume.
2. Frontend sends the file to the backend.
3. Backend checks the file.
4. Backend stores the file in private resume storage.
5. Backend stores the resume metadata in the database.
6. The candidate record is linked to the stored resume.

The resume is not simply placed inside the candidate database row as ordinary text.

4.4 Why Keep Files Separate?

This makes it easier to:

- Manage large files.
- Download resumes when needed.
- Protect private candidate information.
- Control access through backend permissions.
- Keep the database focused on structured information.

---

5. Important Database Tables

The following are the important application entities and their database tables.

Database table	What it stores
`app_users`	Staff login accounts
`clients`	Client portal information
`client_role_assignments`	User roles for each client
`recruitment_processes`	Standard recruitment stage definitions
`client_processes`	Stages enabled for each client
`candidates`	Candidate information
`candidate_stage_records`	Candidate stage decisions and notes
`candidate_events`	Candidate activity history
`candidate_offers`	Offer details and response status
`resume_files`	Resume file metadata
`preonboarding_cases`	Pre-onboarding case information
`preonboarding_access`	Candidate invitation and OTP information
`preonboarding_config`	Pre-onboarding configuration
`preonboarding_settings`	Required details and documents
`preonboarding_documents`	Uploaded document details and review information
`invitation_tokens`	Staff invitation information

Hiring Manager and Requirement entities also have their own database tables using the JPA naming strategy.

5.1 How Tables Are Connected

For example:

A client can have many requirements.

A requirement can have many candidates.

A candidate can have many recruitment stage records.

A candidate can have many activity history entries.

A candidate can have an offer record.

A candidate can have a pre-onboarding case.

A pre-onboarding case can have multiple uploaded documents.

This is how the application connects information without storing everything in one large table.

5.2 Example of a Candidate Record

Imagine the following candidate:

Name: Gowtham S
Email: gowtham@example.com
Client: Weekend Masti
Requirement: Java Developer
Experience: 2 years
Expected CTC: 5 LPA
Notice Period: 30 days

The candidate information is stored in the `candidates` table.

The candidate's stage decisions are stored in `candidate_stage_records`.

The candidate's actions and status changes are recorded in `candidate_events`.

The resume file is stored in private file storage, with its metadata in `resume_files`.

PART 3 – CLIENT AND APPLICATION CREATION

6. How an Admin Creates a Client Application

The Admin can create separate recruitment portals for different clients.

Example:

- Samsung
- Essar

- Weekend Masti

The process:

1. Admin opens Client Management.
2. Admin clicks Create Client.
3. Admin enters the client name.
4. Admin enters the unique portal slug.
5. Admin enters the application name.
6. Admin configures branding.
7. Admin uploads the logo if required.
8. Admin configures recruitment stages.
9. Admin saves the client.

The backend stores the client information in the `clients` table.

The enabled recruitment stages are stored in `client_processes`.

6.1 What Is a Client Slug?

A slug is a unique name used in the portal URL.

Example:

Client name: Weekend Masti

Slug: `wekend-masti`

Portal URL:

`/app/wekend-masti`

The slug helps the application identify which client's portal the user wants to access.

6.2 Client Status and Freeze

The client configuration includes statuses such as:

- DRAFT
- READY
- ACTIVE

There is also a separate frozen setting.

Freezing a client is different from changing its normal status.

When a client is frozen, the application can restrict new applications and changes according to the freeze rules implemented in the backend.

The Admin can unfreeze the client when required.

PART 4 – USER CREATION AND LOGIN

7. How User Creation Works

The Admin manages staff access through User Management.

The Admin can assign users to individual client portals.

Example:

User	Client	Role
recruiter@example.com	Samsung	RECRUITER
recruiter@example.com	Essar	HR_BP
manager@example.com	Weekend Masti	HIRING_MANAGER

The same email can have different roles in different clients.

The database maintains this assignment using:

`client_role_assignments`

The unique combination is:

Email + Client ID

This means the same email cannot have two separate role assignments for the same client.

7.1 Staff Invitation

When an Admin creates or assigns a user:

1. Backend saves the role assignment.
2. Backend generates an invitation token.
3. The token is stored securely.
4. The user receives an invitation email.

5. The user opens the invitation link.
6. The user sets a password.
7. The invitation is accepted.
8. The account can be used to log in.

The default invitation expiry is 24 hours.

---

8. How Login Works

The application uses JWT-based authentication.

JWT means JSON Web Token.

A JWT is a digital token used to identify an authenticated user when they access protected application features.

Login Process

1. User opens the login page.
2. User enters email and password.
3. User selects or enters the client portal where required.
4. Frontend sends the login information to the backend.
5. Backend checks the credentials.
6. Backend identifies the user's role for the selected client.
7. Backend generates an authentication token.
8. Frontend receives the token.
9. Frontend uses the token when calling protected APIs.
10. Backend validates the token for future requests.

The configured JWT expiry is 12 hours by default.

8.1 How Does the Backend Know the User's Role?

The backend checks:

- User email
- Selected client
- Client role assignment

The `RoleResolver` resolves the correct role for the selected client.

For example:

The same person might be:

- RECRUITER in Samsung.
- HR_BP in Essar.

The backend must use the assignment for the selected client, not assume the person has the same role everywhere.

8.2 How Is Admin Different?

ADMIN is a system-level role.

The Admin has centralized application management permissions.

Client-specific roles do not automatically grant Admin permissions.

8.3 How Does the Application Protect Pages?

The frontend uses route guards to control which pages users can open.

The backend also checks permissions when API requests arrive.

This is important because hiding a button in the frontend is not enough to protect information.

The backend is the final authority for protected operations.

PART 5 – ROLE AND PERMISSION SYSTEM

9. What Each Role Can Do

9.1 ADMIN

Admin manages the overall system.

Main responsibilities:

- Create clients.
- Update clients.
- Freeze and unfreeze clients.

- Configure client recruitment stages.
- Create and manage users.
- Assign client roles.
- Manage requirements and candidates.
- Access administration settings.
- Manage permitted offer and pre-onboarding operations.

9.2 RECRUITER

Recruiter manages recruitment operations.

Main responsibilities:

- Create candidates.
- Upload resumes.
- Perform resume screening.
- Manage recruiter-owned recruitment stages.
- Add permitted remarks.
- Manage candidate information.
- Manage permitted client settings.

Recruiters cannot directly edit Hiring Manager Review or Offer Released.

9.3 HIRING_MANAGER

Hiring Manager reviews candidates for assigned requirements.

Main responsibilities:

- View assigned candidates.
- Review candidates after screening has proceeded.
- Update Hiring Manager Review.
- Approve, reject, or hold where permitted.

A Hiring Manager cannot edit unrelated recruitment stages.

9.4 MANAGER

Manager has view-oriented permissions.

The current stage permission configuration gives Manager no stage-edit permissions.

Manager can view candidates and use permitted comment functionality.

9.5 HR_BP

HR BP manages HR-related recruitment stages.

The configured stage list includes:

- HR Round
- Compensation Discussion
- Background Verification
- Offer Released

HR BP also has permissions for offer-related operations and pre-onboarding according to the relevant backend rules.

9.6 CLIENT_ADMIN

Client Admin manages permitted client portal settings.

For example:

- Client branding.
- Site configuration.

Client Admin does not automatically receive candidate stage approval permissions.

10. How Stage Permissions Are Checked

The backend uses `StagePermissionService`.

This service checks:

1. Who is logged in?
2. Which client does the user belong to?
3. Which recruitment stage is being changed?
4. Does this role own that stage?
5. Is the stage currently active?
6. Is the candidate on hold?
7. Is the Hiring Manager assigned to the requirement?

Only when the permission checks pass can the operation continue.

PART 6 – REQUIREMENT AND CANDIDATE MANAGEMENT

11. How a Requirement Is Created

A requirement represents a job opening.

Example:

Position: Java Developer
Experience: 2–4 years
Location: Chennai
Openings: 5
Client: Weekend Masti

Process:

1. Admin or Recruiter opens Requirement Management.
2. User selects the client.
3. User enters job details.
4. User assigns a Hiring Manager if required.
5. User saves the requirement.
6. Backend stores the requirement.
7. The requirement becomes available in the relevant client portal.

12. How Candidates Apply

The application provides a public client job application flow.

1. Candidate opens the client's public recruitment portal.
2. Candidate views available jobs.
3. Candidate selects a job.
4. Candidate enters application details.
5. Candidate uploads a resume.
6. Frontend sends the application to the backend.
7. Backend validates the application.
8. Backend creates the candidate record.

9. Candidate is connected to the relevant client and requirement.
10. Resume is stored separately.

Public application endpoints are available under:

`/api/public/clients`

13. How Recruiters Add Candidates

A Recruiter can also create candidates from the internal portal.

The Recruiter:

- Selects a requirement.
- Enters candidate details.
- Uploads the resume.
- Saves the candidate.

The backend validates permissions and stores the candidate information.

PART 7 – RECRUITMENT STAGE WORKFLOW

14. How Recruitment Stages Work

The recruitment process is divided into stages.

Each stage represents one step in evaluating a candidate.

Example:

Candidate Application → Resume Screening → Hiring Manager Review → Technical Round → Manager Round → HR Round → Offer

The client can configure enabled stages and their display order.

14.1 Stage Decisions

Each stage can have one of the following decisions:

Decision	Meaning
PENDING	No final decision yet
APPROVED	Candidate passed the stage
REJECTED	Candidate did not pass
HOLD	Candidate is temporarily stopped

14.2 Stage Records

The `candidate_stage_records` table stores information about stage decisions.

It includes:

- Candidate
- Recruitment process stage
- Decision
- Notes
- Rating, where applicable
- Updated by
- Updated time

14.3 Candidate History

The `candidate_events` table stores candidate activity.

Examples:

- Candidate created.
- Screening updated.
- Stage approved.
- Candidate rejected.
- Candidate put on hold.
- Candidate resumed.
- Offer accepted.
- Pre-onboarding submitted.
- Document correction requested.

- Pre-onboarding approved.

This provides a history of important candidate actions.

15. Resume Screening

Resume screening is an early recruitment step.

The Recruiter checks:

- Experience
- Relevant experience
- Current CTC
- Expected CTC
- Notice period
- Skills
- Resume

Possible screening outcomes:

PENDING – Screening is not finished.

PROCEEDED – Candidate can move to the next configured stage.

ON_HOLD – Candidate is temporarily stopped.

REJECTED – Candidate is rejected during screening.

Only candidates with PROCEEDED screening status can move to Hiring Manager Review.

16. Hold and Resume

Hold is used when a candidate cannot continue immediately.

Example:

A Hiring Manager needs additional information before making a decision.

The Hiring Manager puts the candidate on hold.

While the hold remains active:

- Candidate workflow is blocked.
- Other stages cannot be changed.

- Only the owner of the held stage can resume or make the permitted decision.

The current implementation defines different hold owners depending on the stage.

For example:

- Hiring Manager Review → HIRING_MANAGER.
- Offer Released → HR_BP.
- Other configured stages → RECRUITER.

The backend checks the hold owner before allowing the action.

PART 8 – EMAIL SYSTEM

17. How Email Works

The application uses JavaMailSender to send emails.

Email is sent by the backend, not directly by Angular.

The backend uses SMTP configuration.

SMTP is the standard method used by applications to send emails through an email server.

Email Flow

1. A user performs an action.
2. Frontend sends the request to the backend.
3. Backend validates the action.
4. Backend prepares the email.
5. Backend connects to the configured SMTP server.
6. SMTP server sends the email.
7. Recipient receives the email.

17.1 Important Email Types

The application sends emails for:

- Staff invitations.
- Access granted notifications.
- Offer letter release.

- Offer response-related communication.
- Pre-onboarding invitations.
- OTP verification.
- Correction requests.
- Pre-onboarding approval.
- Pre-onboarding reminders.

17.2 Offer Letter Email

When Offer Released is approved:

1. Backend creates the offer PDF.
2. The PDF is attached to the email.
3. Candidate receives the email.
4. Configured CC recipients can include Hiring Manager and HR BP.
5. The email contains secure offer response links.

17.3 What Happens If Email Fails?

For the offer release flow, the current implementation treats email delivery failure as an error and rolls back the approval transaction.

This avoids recording a successful offer release when the required email operation fails.

Email configuration must therefore be correct before testing offer release.

PART 9 – OFFER MANAGEMENT

18. How Offer Generation Works

Offer generation begins when the candidate reaches Offer Released.

The backend uses the offer letter template.

The template can be configured through the offer template path. If an external template is not configured, the application uses its built-in template.

The backend fills the offer information and generates a PDF using OpenPDF.

Offer Process

1. Candidate reaches Offer Released.
2. HR BP reviews the offer information.
3. HR BP approves Offer Released.
4. Backend generates the offer PDF.
5. Backend creates a secure offer response token.
6. Token hash is stored in the database.
7. Candidate receives the email and PDF.
8. Candidate opens the offer response link.
9. Candidate chooses Accept or Decline.

18.1 Offer Security

The offer response link uses a secure token.

The database stores the token hash rather than the original token.

The default offer expiry is seven days.

18.2 Offer Statuses

Status	Meaning
RELEASED	Offer has been released
ACCEPTED	Candidate accepted
DECLINED	Candidate declined
EXPIRED	Offer response period expired
WITHDRAWN	Offer was withdrawn

18.3 What Happens After Acceptance?

When a candidate accepts:

- Offer status becomes ACCEPTED.
- Candidate status becomes ACCEPTED.
- Offer Accepted stage is completed.

- A pre-onboarding case is created automatically.
- Candidate receives a pre-onboarding invitation.

When a candidate declines:

- Offer status becomes DECLINED.
- Candidate status becomes DECLINED.

PART 10 – OTP AND PRE-ONBOARDING

19. How Pre-Onboarding Starts

Pre-onboarding starts automatically after offer acceptance.

The backend creates a pre-onboarding case and sends the candidate an invitation.

The HR team can configure which details and documents the candidate must submit.

20. How OTP Works

OTP means One-Time Password.

It is a temporary verification code sent to the candidate's registered email.

The purpose is to confirm that the person accessing the pre-onboarding form has access to the registered email account.

OTP Process

1. Candidate opens the pre-onboarding link.
2. Backend checks the invitation token.
3. Candidate requests an OTP.
4. Backend generates a six-digit OTP.
5. Backend stores the OTP hash.
6. Backend emails the OTP.
7. Candidate enters the OTP.
8. Backend hashes and compares the submitted OTP.
9. If it matches and has not expired, verification succeeds.
10. Candidate can access the form.

20.1 OTP Rules

Rule	Current setting
OTP length	6 digits
OTP expiry	5 minutes
Resend cooldown	45 seconds
Maximum incorrect attempts	5 per issued OTP

The backend stores the OTP hash, not the plain OTP.

The invitation token is also stored as a hash.

The invitation expiry is tied to the configured submission deadline.

21. Candidate Pre-Onboarding Form

After OTP verification, the candidate can complete the form.

The form contains the personal details and document items configured for that client.

Examples of possible information:

- Personal details
- Date of birth
- Current address
- Permanent address
- Contact details
- Education details
- Employment information
- Required identity documents

The actual form contents depend on the client's saved configuration.

Submission Process

1. Candidate enters the required details.
2. Candidate uploads the required documents.
3. Frontend submits the information.

4. Backend validates mandatory fields.
5. Backend checks required documents.
6. Backend saves the information.
7. Case status becomes SUBMITTED.

Uploaded documents are stored in private file storage.

Supported formats:

- PDF
- JPG
- PNG

Maximum file size: 10 MB.

22. HR Document Review

After submission, HR can review the candidate's information.

HR can:

- View submitted details.
- Download uploaded documents.
- Approve individual documents.
- Mark documents as needing correction.
- Request overall correction.
- Perform final approval.

Document Review Decisions

APPROVED – Document is accepted.

NEEDS_CORRECTION – Candidate must correct or replace the document.

22.1 Request Correction

When a correction is required:

1. HR enters a correction comment.
2. Backend saves the correction request.
3. Case status becomes CORRECTION_REQUIRED.

4. Candidate receives a correction email.
5. Candidate opens the secure link.
6. Candidate verifies using OTP.
7. Candidate corrects the information or uploads a replacement document.
8. Candidate submits again.
9. HR reviews the updated submission.

When a document is replaced, the previous document record is marked SUPERSEDED.

23. Final Pre-Onboarding Approval

HR can approve the complete pre-onboarding case only when:

- The case has been submitted.
- All enabled mandatory documents have been approved.
- No documents remain marked NEEDS_CORRECTION.

After approval:

1. Case status becomes COMPLETED.
2. Candidate activity history is updated.
3. Candidate receives an approval email.
4. HR can contact the candidate regarding the next steps.

Important: This is pre-onboarding approval. It does not automatically mark the candidate as JOINED.

PART 11 – STATUS EXPLANATION

24. Candidate Statuses

The current CandidateStatus enum contains:

Status	Meaning
ACTIVE	Candidate is in an active recruitment process
ON_HOLD	Candidate is currently on hold

Status	Meaning
REJECTED	Candidate has been rejected
COMPLETED	Candidate has completed the configured workflow
OFFERED	Candidate has an offered status
ACCEPTED	Candidate has accepted the offer
DECLINED	Candidate has declined the offer

The current enum does not contain JOINED.

25. Pre-Onboarding Statuses

Status	Meaning
CREATED	Pre-onboarding case created
INVITED	Invitation sent
IN_PROGRESS	Candidate is working on the form or documents
SUBMITTED	Candidate submitted the information
CORRECTION_REQUIRED	Candidate must make corrections
COMPLETED	HR approved the pre-onboarding case
EXPIRED	Invitation or submission period expired

26. Offer Statuses

Status	Meaning
RELEASED	Offer sent to candidate
ACCEPTED	Candidate accepted
DECLINED	Candidate declined
EXPIRED	Offer expired

Status	Meaning
WITHDRAWN	Offer withdrawn

Candidate status, offer status, and pre-onboarding status are separate values. They should not be treated as the same thing.

PART 12 – APPLICATION CONFIGURATION

27. Backend Configuration

Main configuration file:

`recruitment-backend/src/main/resources/application.properties`

Important settings include:

- Server port
- Database connection
- JWT secret and expiry
- Admin account configuration
- Email SMTP settings
- File storage directories
- Frontend URL
- CORS allowed origins
- Offer template path
- Offer expiry
- Invitation expiry
- Pre-onboarding configuration

Sensitive values should be provided using environment variables.

Never commit production passwords or secrets to Git.

28. Scheduled Tasks

The application includes a pre-onboarding reminder job.

The current default schedule is daily at 9 AM.

It is responsible for processing eligible pre-onboarding reminders.

The exact reminder behavior depends on the configured service logic.

PART 13 – API AND TECHNICAL COMMUNICATION

29. What Is an API?

An API is how the frontend communicates with the backend.

For example:

When the user clicks Save Candidate:

Angular sends a request to a backend API.

Spring Boot receives it, validates it, saves the information, and returns a response.

The local backend API base URL is:

`http://localhost:8080/api`

Main API Groups

API group	Purpose
`/api/auth`	Login and authentication
`/api/clients`	Client management
`/api/users`	User management
`/api/clients/{clientId}/roles`	Client role assignments
`/api/requirements`	Requirement operations
`/api/candidates`	Candidate operations
`/api/site`	Client portal management
`/api/public/clients`	Public client pages and applications
`/api/public/offers`	Offer response
`/api/public/preonboarding`	Candidate OTP and pre-onboarding

Refer to `API_REFERENCE.md` for the complete endpoint list.

PART 14 – INSTALLATION AND RUNNING

30. Required Software

Before running the project, install:

- Java 17
- Maven
- Node.js
- npm
- MySQL
- Git

31. Database Setup

Create the database:

```
CREATE DATABASE recruitment_db;
```

Configure database credentials using environment variables.

The application uses Hibernate with `ddl-auto=update` in the supplied configuration.

This can create or update mapped database tables automatically during development.

For production, use a controlled database migration strategy and take backups before schema changes.

32. Start Backend

```
cd recruitment-backend
mvn clean install
mvn spring-boot:run
```

Backend:

`http://localhost:8080`

33. Start Frontend

Open another terminal:

```
cd recruitment-admin
npm install
npm start
```

Frontend:

`http://localhost:4200`

PART 15 – IMPORTANT SECURITY RULES

34. Application Security

The application handles private candidate and employee information.

Important rules:

1. Do not share production database passwords.

2. Do not commit JWT secrets.
3. Do not expose private resume directories publicly.
4. Do not expose pre-onboarding documents through public URLs.
5. Do not store plain OTPs.
6. Do not store plain invitation tokens.
7. Do not trust frontend permission checks alone.
8. Always validate client access in the backend.
9. Validate uploaded document formats and sizes.
10. Use HTTPS in production.
11. Change development default passwords.
12. Back up the database and private files.

PART 16 – COMPLETE APPLICATION FLOW

35. End-to-End Recruitment Example

Imagine a company creates a Java Developer requirement.

Step 1 – Client Creation

Admin creates the client portal.

Step 2 – User Assignment

Admin assigns:

- Recruiter
- Hiring Manager
- HR BP
- Manager

Step 3 – Requirement Creation

Recruiter creates a Java Developer requirement.

Step 4 – Candidate Application

Candidate applies through the public portal or Recruiter adds the candidate internally.

Step 5 – Resume Screening

Recruiter reviews the resume and candidate information.

Step 6 – Hiring Manager Review

If screening proceeds, the candidate can move to Hiring Manager Review.

Step 7 – Technical and Other Interviews

The candidate moves through the enabled technical, functional, manager, and HR stages.

Step 8 – Offer Release

HR BP approves Offer Released.

The backend generates the offer PDF and sends it by email.

Step 9 – Offer Response

Candidate opens the secure link and accepts or declines.

Step 10 – Pre-Onboarding Invitation

After acceptance, the backend automatically creates the pre-onboarding case and sends the invitation.

Step 11 – OTP Verification

Candidate requests and enters the email OTP.

Step 12 – Candidate Information

Candidate enters required personal details and uploads documents.

Step 13 – HR Review

HR reviews the submitted information and documents.

Step 14 – Correction, If Required

If something is incorrect, HR requests correction.

Candidate updates the information and submits again.

Step 15 – Final Approval

HR approves the pre-onboarding case after all mandatory documents are approved.

Candidate receives an approval email.

Step 16 – Joining

HR contacts the candidate regarding the next steps and actual joining.

Pre-onboarding approval and joining must remain separate business events.

PART 17 – IMPORTANT NOTES FOR FUTURE DEVELOPERS

36. Before Making Changes

Before changing any module:

1. Understand the existing workflow.
2. Check the related frontend component.
3. Check the backend controller.
4. Check the service implementation.
5. Check the entity and repository.
6. Check the role permissions.
7. Check the database impact.
8. Check email behavior if the feature sends emails.
9. Test the complete flow.
10. Update documentation.

37. When Adding a New Recruitment Stage

Check:

- Recruitment process definition.
- Client stage configuration.
- Frontend stage display.
- Backend stage permissions.

- Stage decision handling.
- Candidate status changes.
- Candidate history.
- Hold and resume behavior.
- Tests.

38. When Adding a New User Role

Check:

- Role enum.
- User assignment logic.
- Login and role resolution.
- Frontend route guards.
- Frontend permission checks.
- Backend security configuration.
- Stage permissions.
- Client access restrictions.
- Tests.

39. When Adding a New Email

Check:

- Which action triggers the email.
- Which service prepares it.
- Who receives it.
- Whether CC is required.
- Whether attachments are required.
- What happens if email sending fails.
- Whether the action should be rolled back if sending fails.